

Class 9: Dealing with Data — pandas Basics

Data-driven Systems Engineering

Agenda

- Loading a dataset and understanding the pandas Index.
- Selecting rows and columns: `loc`, `iloc`, and boolean filters.
- Renaming and dropping columns for clarity.
- Categorical summaries and grouped statistics.
- Where “cleaning” ends and feature engineering begins.

Learning goals

- Load a dataset and inspect its structure, types, and missingness.
- Select data precisely using label-based, position-based, and boolean access.
- Rename and drop columns to make a DataFrame easier to work with.
- Summarize a categorical column and compute grouped statistics.
- Explain the boundary between pandas-level cleaning and feature engineering.

Course Map and Running Cases



Today in the course

Focus: pandas mechanics for loading, indexing, and lightly cleaning a dataset.

Bridge: Class 6 got data into a DataFrame; this class is about handling that DataFrame with precision before anything gets modeled.

Fraud case

The fraud dataset's target column arrives named `Class`; renaming it to `is_fraud` and separating it from the features is pandas mechanics, not modeling.

Pasteurization case

The pasteurization case eventually needs the same discipline: consistent column names and dtypes before any sensor batch can be trusted.

Course thread: This class deliberately stops at cleaning. Deciding which features to derive or transform is data-theory work, picked up later in the Data-Driven block.

Loading and a first look

```
iris = sns.load_dataset("iris")  
iris.head()  
iris.info()
```

- From 'notebooks/Class2_EDA.ipynb': `iris` is a `DataFrame` with columns `sepal_length`, `sepal_width`, `petal_length`, `petal_width`, and `species`.
- Every `DataFrame` carries a row **Index** alongside its columns — by default, integers starting at 0.
- `.info()` reports the dtype of every column: four `float64` columns and one object column here.

Checking for missing values

```
iris.isnull().sum()
```

- Also from 'Class2_EDA.ipynb': this turns "is anything missing?" into one concrete number per column.
- Iris happens to be clean — every count is zero — but the habit matters more than this particular answer.
- The fraud dataset needs the same check before its 30-plus columns are trusted for anything.

Selecting with loc and iloc

```
iris.loc[0, "species"]          # label-based: row label 0, column "species"  
iris.iloc[0, 4]                 # position-based: first row, fifth column  
iris.loc[:, ["species", "petal_length"]] # a subset of columns
```

- `.loc` indexes by label (row label and column name); `.iloc` indexes by integer position, regardless of label.
- For a DataFrame with the default integer index, label and position often coincide for rows — but they are not the same concept, and a filtered DataFrame makes the difference obvious.

Boolean filtering

```
setosa_only = iris[iris["species"] == "setosa"]  
large_petals = iris[iris["petal_length"] > 5.0]
```

- A boolean expression inside `df[...]` keeps only the rows where the condition is true.
- This is the single most common way to answer “show me the subset where...” in pandas.
- The same pattern isolates fraud cases in the credit-card dataset, or a single process state in the pasteurization data.

Renaming and dropping columns

```
df.rename(columns={'Class': 'is_fraud'}, inplace=True)

X = df.drop('is_fraud', axis=1)
y = df['is_fraud']
```

- From 'streamlit/modeling.py': the raw dataset calls its target Class; renaming it to is_fraud makes every later line more readable.
- inplace=True modifies df directly instead of returning a new, renamed copy.
- Splitting a target out with .drop() and [...] is still pandas mechanics — deciding *which columns to keep* as features is feature engineering, not this class.

```
iris["species"].value_counts()
```

- From 'Class2_EDA.ipynb': one line that turns a categorical column into a count per category.
- For iris the classes are balanced (50 each); for the fraud target, the same call reveals a dataset that is almost entirely non-fraud — a fact any later modeling decision has to respect.

Grouped statistics

```
iris.groupby("species").describe().T
```

- Also from 'Class2_EDA.ipynb': `groupby` splits the DataFrame by category, then `.describe()` summarizes each group separately.
- `.T` transposes the result so species become columns and statistics become rows — purely a readability choice.
- This single line already tells you more than a global `.describe()` ever could, without touching a model.

Duplicates and obvious dtype fixes

- `df.duplicated()` flags exact-duplicate rows; `df.drop_duplicates()` removes them.
- `df["col"].astype(...)` fixes a column stuck in the wrong dtype, for example a numeric column read in as text.
- These are still mechanical fixes: they correct how the data is represented, not what the data should mean for a model.
- Always check row counts before and after — a silent drop of “duplicate” rows that are not really duplicates is a common mistake.

Cleaning versus feature engineering: where this class stops

This class (cleaning)	The Data-Driven block (feature engineering)
Renaming a confusing column	Deciding which columns become model inputs
Checking and handling missing values	Imputation strategy and its trade-offs
Fixing an obviously wrong dtype	Encoding, scaling, and transformation choices
Dropping an exact duplicate row	Deriving new columns from existing ones

- Everything on the left is pandas mechanics; everything on the right is a modeling decision with theory behind it.

How the repo already uses today's building blocks

Repo activity

- 'notebooks/Class2_EDA.ipynb' runs `.head()`, `.info()`, `.isnull().sum()`, `.value_counts()`, and `.groupby(...).describe()` on the iris dataset in sequence.
- 'streamlit/modeling.py' renames the fraud target column before ever splitting features from labels.
- Open the notebook and identify which of today's five operations appears in which cell.

Question

'value_counts()' on the fraud target shows a dataset that is almost entirely non-fraud. Is that a data-cleaning problem, a feature-engineering problem, or neither?

- Deliberately has no clean answer yet — it previews why class imbalance gets its own treatment later in the Data-Driven block.

Papers and materials

pandas documentation; pandas overview; Sambasivan et al. (2021), Everyone wants to do the model work, not the data work.

From This Class to the Next

What stays with us

- `loc`, `iloc`, and boolean filters cover almost every “give me this subset” question you will ask of a `DataFrame`.
- Renaming, dropping, and checking dtypes are cleaning; deciding what a feature should be is a different, later decision.
- ‘`Class2_EDA.ipynb`’ and ‘`modeling.py`’ already show every operation from today in a real dataset.

Project use

Any project step that starts with “first, load and clean the data” is exactly today’s material, no more and no less.

Next connection

Class 10 uses these same habits to run a full exploratory data analysis, this time reading plots as well as tables.

Course thread: Clean, well-indexed data is what makes the next class’s plots worth trusting.